TrueRepairs

CSC 4996 – Test Plan

HISHAM ISSA, ROBERT SIMOVSKI, ISHMAEL GONZALES, JIBRAN IQBAL

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Objectives 4

1.3 Scope 4

1.4 Audience 4

2. Test Objectives 5

3. Test Strategy 5

3.1 Testing Levels 6

3.2 Testing Tools 7

4. Test Scope 8

4.1 In-Scope Components 8

4.2 Out-of-Scope 8

5. Test Deliverables 8

6. Test Schedule 9

7. Test Environment 10

7.1 Hardware 10

7.2 Software 10

7.3 Environmental Conditions 11

8. Test Entry Criteria 11

8.1 Environment Setup 11

8.2 Core Feature Readiness 11

8.3 Exit Criteria 12

9 Test Pass/Failure Criteria 13

9.1 Test Pass Criteria 13

9.2 Test Fail Criteria 13

10 Test Suspension/Resumption Criteria 14

10.1 Suspension Criteria 14

10.2 Resumption Criteria 14

11. Test Design & Execution Approach 15

11.1 Test Design Approach 15

11.2 Test Execution Approach 15
12. Test Data 16
12.2 Vehicle Data 17
12.3 AI Detection Files 17
12.4 Report & Estimate Data 17
12.5 Backend Payload Examples 17
13 Risk Analysis 18
13.1 Technical Risks 18
13.2 Environmental Risks 19
13.3 Human Risks 19
13.4 External Risks 19
14 Defect Management Process 20
14.1 Defect Reporting 20
14.2 Defect Severity Levels 20
14.3 Defect Lifecycle 20
15 Test Cases 21
16 Roles and Responsibilities 74
17 Test Report 75

1. Introduction

1.1 Purpose

The purpose of the TrueRepairs Test Plan is to define the structured process used to verify and validate the functionality, performance, and reliability of the TrueRepairs web application. This document outlines how the testing team will confirm that all system components operate as expected and meet the requirements outlined in the Software Requirements Specification (SRS). The Test Plan ensures that the system is evaluated consistently, defects are identified early, and the final product delivers a dependable experience for users.

1.2 Objectives

The objective of this Test Plan is to establish a controlled and repeatable testing process for TrueRepairs. Testing activities will verify that the application functions correctly across all major features including account registration, login, vehicle entry, VIN decoding, AI detection, manual report building, estimate generation, and report output. The objectives also include validating performance expectations, ensuring system reliability, and confirming that external integrations behave properly under normal and failure conditions.

1.3 Scope

The scope of testing includes all frontend and backend components developed by the TrueRepairs team. This includes user authentication, dashboard features, vehicle management, estimate generation, AI-based part detection, manual report creation, and account settings. Out of scope are external third-party services such as RockAuto data files, NinjaAPI, OpenAI API, Cloudinary, and third-party UI libraries. These systems are assumed to have been tested by their providers.

1.4 Audience

This document is intended for the TrueRepairs development team, quality assurance testers, and stakeholders. Developers will use this document to understand the testing scope, review expected outcomes, ensure their code meets requirements, and address any defects identified during testing. QA testers will use this document to design and execute test cases, document the results, track defects for fixing, and verify that all system requirements are properly validated. Stakeholders will use this document to assess the testing approach, review overall completeness of the application, monitor testing progress, and support decision making regarding the system's readiness for release. This test plan provides detailed guidance for executing tests, analyzing results, and confirming that all requirements have been validated before the final release.

2. Test Objectives

The objective of testing the TrueRepairs system is to verify that all major features function as expected, meet the requirements defined by the development team, and provide a reliable, usable experience. Testing will confirm both functional and non-functional aspects—including performance, usability, error handling, resilience, and external integrations (OpenAI, JSON parts data, APIs, and PDF generation)—operate correctly without introducing excessive delays.

Successful testing will demonstrate that TrueRepairs meets its requirements and is ready for prototype demonstration and deployment.

3. Test Strategy

TrueRepairs will use a combined **black-box** and **gray-box** testing approach.

1) Black-Box Testing

- UI/UX behavior (buttons, popups, navigation, responsive design)
- Input validation (invalid VIN, wrong login credentials, missing fields)
- End-to-end workflows, including:
    - Account registration & login
    - Vehicle management
    - AI damage scan flow
    - Manual report creation
- Report generation & download

Ensures the application works correctly for users regardless of how the internals are implemented.

2**) Gray-Box Testing**

- API endpoint validation
    - Correct request/response structure
    - Proper HTTP status codes
    - Successful and failure scenarios
- Backend processing logic
    - AI scan integration logic
    - Estimate calculations
    - Data transformations for reports
- Data flow correctness
    - Frontend > Backend > Database
    - Ensuring the stored data matches expected formats
- Security checks

- JWT authentication flows
- Role-based access
- Input sanitization

This hybrid approach ensures both functional coverage and internal logic validation.

3.1 Testing Levels

Unit Testing

- Tests individual backend functions (Node/Express controllers)
- Tests frontend components in isolation
- Ensures modules behave correctly before integration

Integration Testing

- Verifies interaction between:
  - Frontend - Backend APIs
  - Backend - MongoDB
- Example: Login generates JWT and unlocks the dashboard.

System Testing

End-to-end workflow testing of registering an account, creating a vehicle profile, generating a manual report, and downloading it as a PDF report to validate all major features:

- Account Creation:
  - User registers by filling out the registration form with valid credentials. The system sends a verification email.
    Expected Result: User receives the email, and clicking the verification link activates the account.
- Login:
  - User logs in with the registered email and password.
    Expected Result: User successfully logs in and is redirected to the dashboard.
- Add Vehicle Profile:
  - User clicks “My Vehicle” on the dashboard. In the My Vehicle popup, the user can create a vehicle profile either by entering a VIN or manually providing make, model, and year.
    Expected Result: Vehicle profile is saved and displayed in the My Vehicle history section.
- Generate Manual Repair Report:
  - User selects “Manual Report” on the dashboard and chooses a vehicle profile along with the required options to generate the report.
    Expected Result: Manual repair report is created correctly and appears in the history section of the Manual Report popup on the right.
- Generate Ai Repair Report:

    - User selects the Smart Report option on the dashboard, chooses a vehicle profile, enters a labor rate, and uploads vehicle images.
    - Expected Result: The system utilizes AI to detect parts and generate the repair report, which appears in the history section of the Smart Report popup on the right.
- Download Repair Report as PDF:
    - User clicks the download button for a selected report.
      Expected Result: PDF file downloads successfully, containing all relevant information.

Acceptance Testing

- User-oriented tests confirming readiness for final demonstration
- Uses real-world shop scenarios to validate smooth workflows:
    - Realistic Vehicle Data:
        - Test with real VINs from common vehicles (e.g., 2006 Toyota Camry) to ensure the system correctly creates vehicle profiles.
        - As an alternative path, use real make/model/year combinations when manually creating vehicle profiles.

- Actual Repair Scenarios
    - Manual Report:
        - User selects realistic vehicle damage scenarios using the "Area of Car" dropdown (e.g., engine, electrical, body & lamp assembly).
        - User selects real parts typically repaired in shops (e.g., front bumper, rear bumper, fender).
        - Expected Result: The report reflects accurate, shop-realistic repair items.

- Smart Report (AI-based):
    - User uploads real car damage images taken in shop-like conditions.
    - AI should detect actual vehicle parts (e.g., cracked bumper, damaged door panel).
    - User enters a realistic labor rate (e.g., $110/hour).
    - Expected Result: AI generates a repair report consistent with what a repair shop would produce.

3.2 Testing Tools

- **Postman** Used only to test the API endpoints that our team developed (/register, /login, /deleteAccount).
- **MongoDB Compass** to inspect stored data during tests

- **Browser DevTools** for frontend debugging
- **GitHub Issues** for defect tracking
- **Manual testing** for full UI verification
- **Automated scripts** for small, targeted unit tests (limited)

4. Test Scope

4.1 In-Scope Components

Frontend

- Registration and Login
- User Profile (update email, update password)
- Vehicle Entry (manual + VIN decode)
- AI Detection view
- Manual Report builder
- Estimate summary + PDF export
- Account Settings (deactivate, delete)

Backend

- Endpoints for:
    - Registration/Login
    - Account deletion & deactivation o Vehicle creation
    - Vehicle history retrieval
    - Report generation
    - AI detection workflow
- Email sending (verification)
- PDF generation logic
- Database validation & storage

4.2 Out-of-Scope

External services assumed to be fully tested by their providers:

- OpenAI API
- Cloudinary image storage
- NinjaAPI
- RockAuto JSON data files

5. Test Deliverables

The testing process will generate the following deliverables:

1) **Test Plan Document**

This formal document describing testing approach and coverage.

2) **Test Cases**
Complete list of all frontend and backend test cases (TC).

3) **Test Scripts**
Postman collections and sample JSON payloads.

4) **Test Data Files**
Sample VINs, emails, JSON parts lists, PDF output samples.

5) **Defect Reports (GitHub Issues)**
Logged defects with severity, steps, screenshots, and status.

6) **Test Execution Report**
Summary of pass/fail results for all cases.

7) **Traceability Mapping (if needed)** *Links requirements to test cases – located in xlsx file*.

8) **Final Approval Record**
Confirmation that testing has been completed and accepted.

6. Test Schedule

Date	Test Executor	Test Type	Test Cases
10/10 - 10/15	Ishmael	Integration	TC-01 - TC-10
10/17 - 10/ 23	Hisham	Integration	TC-11 - TC-20
10/25 - 10/30	Robert	Integration	TC-21 - TC-30
11/01 - 11/05	Ishmael	Integration	TC-31 - TC-38
11/07 - 11/11	Jibran	Unit	TC-39 - TC-41
11/13 - 11/15	Jibran	Unit	TC-42 - TC-44

11/16 - 11/20	Robert	System	TC-45 - TC-47
11/25 - 12/02	GTA	Acceptance	TC-49 - TC-51

7. Test Environment

7.1 Hardware

- Developer laptops (Windows 11, MacOS Ventura)
- Smartphones for mobile browser testing (IOS 14, Android 13)

7.2 Software

Frontend:

- React.js v19.1.10
- Vite v7.1.2
- Chrome v80, Edge v142, Firefox v87

Backend:

- Node.js v22.17.1
- Required npm packages
    - @sendgrid/mail v8.1.6
    - Axios v1.12.2
    - Bcrypt v6.0.0
    - Bcryptjs v3.0.2
    - Cloudinary v2.8.0
    - Cors v2.8.5
    - Dotenv v17.2.3
    - Express v5.1.0
    - Jsonwebtoken v9.0.2
    - Lucide-react v0.546.0
    - Mongoose v8.18.2
    - Multer v2.0.2
    - Nodemon v3.1.10
    - Openai v6.9.1
    - Streamifier v0.1.1

Database:

- MongoDB Atlas v6.19.0
- MongoDB Compass v1.47.0

Tools:

- Postman v11.73.5
- VS Code v1.106.3
- GitHub v3.5.4
- Browser DevTools (included with the browsers listed above)

7.3 Environmental Conditions

- Stable internet connection
- Database seeded with test data
- Browser tests under various resolutions
- Light & dark mode compatibility checks

8. Test Entry Criteria

Testing for TrueRepairs will begin only when the following conditions are met:

8.1 Environment Setup

- Backend (Node/Express) is running with no critical errors
- Frontend (React + Vite) builds successfully and loads without failures
- MongoDB Atlas is connected and seeded with test data
- All required API keys and environment variables are configured:
  - JWT secret
  - MongoDB URI
  - OpenAI
  - Credentials (gcv.json)
  - SendGrid API key
  - Toyota/RockAuto JSON data

8.2 Core Feature Readiness

- Account registration and login fully functional
- Password reset and verification features working
- Vehicle entry (manual + VIN decode) implemented
- AI part detection connected to OpenAI
- Estimate generation and PDF export functional
- No blocker-level bugs in critical flows

Smoke test pass requirement is to verify that the main functionality of TrueRepairs works correctly. The test process follows this sequence:

1. Login: usage of valid credentials

2. Add Vehicle Profile: VIN or Manually
3. Generate Report: Manually
4. Generate report: AI
5. Upload Vehicle Image (when using generating report with AI)
6. Download PDF repair report

When all steps are completed without errors, the application is ready for full testing.

8.3 Exit Criteria

Testing will be considered complete when the following conditions are met:

1. **All major test cases pass:**

    - Authentication testcases: Registration (TC-01), login (TC-08)
    - Vehicle entry test cases: vehicle profile (TC-32)
    - AI detection Smart Report test case: (TC-19)
    - Report generation: Manual Report test cases: (TC-15), Smart Report (TC-18)
    - Account management features test cases: User profile (TC-14), Account Delete (TC-31)

2. **Test cases acceptable for failure:**

    - TC-48: Homepage Load Time – The homepage takes slightly longer than 5 seconds to fully load. This is a non-critical performance issue that does not block functionality or workflow completion.

3. **All critical or high-severity defects are fixed**

    - No blockers remain
    - No crashes or data corruption
    - No broken workflows

3. **Performance tests meet acceptable response times**

- Acceptable response times for the application's main functionality of Login, AI scan, Vehicle Profile, PDF export operates within expected speed limits under normal conditions:
    - Login: authentication process and redirected to dashboard complete within 2 seconds.
    - AI Scan: generate a complete repair report within 10 seconds.
    - Vehicle profile: vehicle profile appears in history section within 2 seconds after entering vehicle information Manually or VIN and clicking save.
    - PDF download: the repair report downloads as a pdf within 1 second

4. **Usability is confirmed**

    - Testers can complete workflows without confusion
    - UI is stable across browsers and resolutions:
        - Usability is confirmed for mobile devices through responsive layout testing on iOS and Android

5. **Documentation is updated**

    - Test results recorded
    - Known issues documented
    - Final approval given by team lead

9 Test Pass/Failure Criteria

9.1 Test Pass Criteria

A test case will be considered a **Pass** if:

- The actual output matches the expected output
- No errors, crashes, or console failures occur
- Data saved in the database is correct and complete
- API responses return the correct status codes and JSON structure
- UI elements display the correct messages and navigation behavior
- Performance meets the expected response time:
    - Login completes within 2 seconds.
    - Smart Report generates a complete repair report within 10 seconds.
    - Vehicle profile saves and displays within 2 seconds
    - Download repair report as a pdf within 1 second

9.2 Test Fail Criteria

A test case will be considered a **Fail** if:

- Actual results differ from expected results
- Any unexpected exceptions or backend errors occur
- UI/navigation behaves incorrectly
- Wrong or incomplete data is stored in the database
- Performance is significantly slower than expected
- A defect prevents completing the workflow
- Security rules are bypassed (e.g., accessing protected pages without login)

10 Test Suspension/Resumption Criteria

10.1 Suspension Criteria

Testing will be paused if any of the following occurs:

1. **Multiple consecutive failures**
    - 5 or more system-level tests fail in a row
    - Indicates a major regression or broken build
2. **Critical System Failure**
    - Backend server crashes or fails to start
    - Database becomes unreachable or corrupted
    - Frontend fails to compile or load key components
3. **Blocking Defects**
    - A major error prevents testers from completing required workflows
    - Missing API keys, misconfigured .env, or deployment issues
4. **Data Integrity Issues**
    - Corrupted test data
    - Inconsistent or duplicate database records
    - Missing required vehicles or account entries

10.2 Resumption Criteria

Testing will resume when:

1. **Environment is restored**
    - Backend and frontend run normally
    - Database is online and contains valid test data
    - All tools (Postman, DevTools) function correctly
2. **Blocking defects are fixed**
    - Developers resolve issues
    - Unit tests confirm the behavior
    - Core workflows operate normally again
3. **Test data is reset**
    - Clean test accounts created
    - VINs and vehicle profiles restored
    - Reports and history cleared if needed

4. **External dependencies restored**

   - API keys updated
   - OpenAI and MongoDB Atlas functioning

11. Test Design & Execution Approach

The test design for TrueRepairs includes both manual and automated testing techniques to ensure comprehensive coverage of all system features.

11.1 Test Design Approach

- **Requirement-Based Test Design**
  Each test case is derived from the functional and non-functional requirements in the SRS to ensure full coverage.
- **Scenario-Based Testing**
  Realistic automotive repair shop scenarios are used to validate the system's behavior end-to-end.

- **Input Validation Coverage**
  Positive, negative, and boundary input tests are included for forms (registration, login, vehicle entry, settings).

- **API Response Validation**
  Backend endpoints are tested for:

  - Correct status codes
  - Error handling
  - JSON structure
  - Database effects

- **UI & UX Coverage**

  Ensures:

  - Correct form validation
  - Accurate error messages
  - Navigation works
  - No broken components

11.2 Test Execution Approach

Frontend Execution (Manual)

- Access the application through browsers

- Complete user workflows step-by-step
- Validate UI behavior, error messages, and navigation
- Observe console logs and network requests

Backend Execution (Automated + Manual)

- Use Postman collections to run backend API tests
- Validate responses for both success and failure cases
- Inspect changes in MongoDB Compass
- Run repeated tests to ensure consistency

Execution Tracking

- Each test case receives:
- Pass/Fail result
- Notes or screenshots
- Defect number (if failed)

12. Test Data

Testing TrueRepairs requires consistent and controlled datasets to ensure results are repeatable and reliable. The following categories of test data will be used: **12.1 User Accounts**

Email	Password	Purpose
[redacted]	[redacted]	Valid registered user
[redacted]	[redacted]	Email update validation
[redacted]	[redacted]	Report testing user
dropeso temp emails	Mixed	Negative test cases
Unregistered emails	—	Error handling

12.2 Vehicle Data

VIN	Vehicle	Purpose
JTENU5JR7D1234567	2013 Toyota 4Runner	Positive vehicle entry
Fake/invalid VINs	—	Negative validation tests
Blank VIN	—	Boundary testing

12.3 AI Detection Files

- Sample car damage images for OpenAI
- Images with:
- Clear damage
- Multiple damages
- No damage (for false positives)
- Blurry/poor lighting
- Abnormal images:
    - Fully black image
    - Fully white image
    - Distorted image
    - Image too small
    - Image too large
    - Unidentified objects

12.4 Report & Estimate Data

- Manual report templates containing:
    - Originally, the system used two separate report JSON formats (14-part and 12-part). These have now been merged into one unified Part Report JSON to maintain consistency and simplify processing.

- Valid and invalid part arrays:
    - totalLow: the sum of the lowest price for each part in the repair report
    - totalHigh: the sum of the highest price for each part in the repair report.
    - totalAvg: the sum of the average price for each part in the repair report.

12.5 Backend Payload Examples

Valid Registration JSON

```
{
```

```
  "email": [redacted]
  "password": [redacted]
  "passwordConfirm": [redacted]
}
```


Vehicle Entry JSON


```
{
  "vin": "JTENU5JR7D1234567",
  "email": [redacted]
}
```


Manual Report JSON Example


```
{
  "title": "2023 Toyota PRIUS (14 parts)",
  "userEmail": [redacted]
  "vehicle": {...},
  "parts": [...],
  "summary": {
    "totalLow": 1046.97,
    "totalHigh": 1282.98,
    "totalAvg": 1138.11
  }
}
```


13 Risk Analysis

13.1 Technical Risks

Risk	Likelihood	Impact	Mitigation
Database connection failures	Medium	High	Maintain backup DB; clear test collections regularly

API failures	Medium	High	Unit test controllers, validate inputs, rate-limit
Authentication/session issues	Medium	High	Test JWT expiration, invalid tokens, restricted routes
UI rendering issues	Medium	High	Test across browsers, use React DevTools
Performance slowdowns	Low	Medium	Optimize DB queries, index fields, test under load

13.2 Environmental Risks

Risk	Likelihood	Impact	Mitigation
Version mismatch between environments	Medium	Medium	Document setup requirements; use shared .env template
Browser differences	Medium	Medium	Test on Chrome, Edge, Firefox
Network instability	Low	Medium	Allow local offline testing when possible

13.3 Human Risks

Risk	**Likelihood**	**Impact**	**Mitigation**
Team scheduling conflicts	Medium	High	Early planning, backup testers
Incorrect test execution	Low	Medium	Provide detailed steps; review results

13.4 External Risks

Risk	Likelihood	Impact	Mitigation
Hardware failure	Low	High	Sync code to GitHub; backup docs

Third-party API updates	Medium	High	Lock dependency versions; avoid late updates
Third-party API Failure	Medium	High	Retry logic if primary API is unavailable; log error and notify the users of service failure.

14 Defect Management Process

14.1 Defect Reporting

All defects will be reported through **GitHub Issues**, including:

- Test Case ID
- Title
- Severity (Critical, High, Medium, Low)
- Steps to reproduce
- Expected result
- Actual result
- Screenshots/logs
- Assigned developer

14.2 Defect Severity Levels

Level	**Description**
Critical	Blocks major workflow or causes crashes
High	Major feature broken but not entire system
Medium	Incorrect output, missing UI element
Low	Minor UI issue, text error

14.3 Defect Lifecycle

1. **Open** – Tester reports issue
2. **Assigned** – Team lead assigns to developer
3. **In Progress** – Developer working on fix
4. **Resolved** – Fix implemented
5. **Verified** – Tester confirms fix
6. **Closed** – Issue completed

15 Test Cases

Integration Test Cases List:

ID	TC-01
Item to Test & Description	Account Registration – Test the registration process to ensure that a user can create an account with valid information.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the register button on the navbar. • The register popup box is opened.
Test Steps	1. User enters in their credentials in the registration form as follows: • Email Address: [redacted] • Password: [redacted] • Confirm Password: [redacted] 2. User click Create Account button 3. User opens their email inbox

	4. User opens the verification link sent by TrueRepair 5. User is redirected to the home page with a notification stating "Account verified." 6. User clicks the login button. 7. User enters credentials: • Email Address: [redacted] • Password: [redacted] 8. User clicks login button.
Expected Results	• The system creates an account for the user. • Verification email link is sent to user's email • The verification link activates the user's account • After logging in, the system redirects the user to the dashboard page.
Priority	High
Pass/Fail Criteria	Pass: The account is created successfully, the verification email is sent, the verification link activates the account, the user can log in, and the system redirects the user to the dashboard page. Fail: • Account creation fails due to invalid data (e.g., mismatched passwords, invalid email format, empty text fields) • Verification link was not sent • The verification link did not activate the account. • Login failed after the verification process. • System did not redirect the user to the dashboard page

ID	TC-02
Item to Test & Description	Account Registration – Test that the user cannot create an account when the email address field is left empty.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the register button on the navbar. • The register popup box is opened.
Test Steps	1. User enters their credentials into the registration form text fields as follows: • Email: (Leave this field blank) • Password: [redacted] • Confirm Password: [redacted] 2. User click Create Account button
Expected Results	• The system does not create an account for the user • The system displays text under the email address text field stating, “Please fill out this field.” • The user remains on the registration popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not create the account and shows the “Please fill out this field.” message and keep the user on the registration popup box. Fail: The system does not display the error message and allows the user to proceed without entering an email, allowing the user to continue into the account creation process.

ID	TC-03
Item to Test & Description	Account Registration – Test that the user cannot create an account when the password field is left empty.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the register button on the navbar. • The register popup box is opened.
Test Steps	1. User enters their credentials into the registration form text fields as follows: • Email: [redacted] • Password: (Leave this field blank) • Confirm Password: [redacted] 2. User attempts to click the Create Account button.
Expected Results	• The system disables the Create Account button • The system does not create an account. • The system displays an error message stating, “Passwords don’t match.” due to the empty Password field while Confirm Password is filled. • The user remains on the registration popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not create the account, disables the Create Account button, shows the “Passwords don’t match” message, and keeps the user on the registration form. Fail: The system does not display the error message and allows the user to proceed without entering a password, allowing the

	user to continue into the account creation process.

ID	TC-04
Item to Test & Description	Account Registration – Test that the user cannot create an account when the confirm password field is left empty.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the register button on the navbar. • The register popup box is opened.
Test Steps	1. User enters their credentials into the registration form text fields as follows: • Email: [redacted] • Password: [redacted] • Confirm Password: (Leave this field blank) 2. User attempts to click the Create Account button.
Expected Results	• The system disables the Create Account button • The system does not create an account. • The system displays a message showing the password requirements. • The user remains on the registration popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not create the account, disables the Create Account button, displays the password-requirements message, and keeps the user on the registration popup box. Fail: The system does not display any error message and allows the user to proceed

	without entering a confirm password, letting the user continue into the account creation process.

ID	TC-05
Item to Test & Description	Account Registration – Test that the user cannot create an account when the email address, password, and confirm password fields are left empty.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the register button on the navbar. • The register popup box is opened.
Test Steps	1. User enters their credentials into the registration form text fields as follows: • Email: (Leave this field blank) • Password: (Leave this field blank) • Confirm Password: (Leave this field blank) 2. User attempts to click the Create Account button.
Expected Results	• The system disables the Create Account button • The system does not create an account. • The user remains on the registration popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not create the account, disables the Create Account button, and keeps the user on the registration popup box. Fail: The system allows the user to proceed without entering an email address, password,

	and confirm password, letting the user continue into the account creation process.

ID	TC-06
Item to Test & Description	Account Registration – Test that the user cannot create an account when an invalid email address format is used.
Pre-Conditions	• The website is running normally, and all essential services are operational. • The user clicks on the register button on the navbar. • The register popup box is opened.
Test Steps	1. User enters their credentials into the registration form text fields as follows: • Email: bent@.com • Password: [redacted] • Confirm Password: [redacted] 2. User clicks the Create Account button.
Expected Results	• The system does not create an account. • The system displays an error message indicating the email address is invalid. • The user remains on the registration popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not create the account, displays the appropriate validation error message, and keeps the user on the registration popup box. Fail: The system allows the account to be created or does not display an invalid email error message.

ID	FTC-07
Item to Test & Description	Account Registration – Test that the user cannot create an account when using an already registered email address.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the register button on the navbar. • The register popup box is opened.
Test Steps	1. User enters their credentials into the registration form text fields as follows: • Email: [redacted] • Password: [redacted] • Confirm Password: [redacted] 2. User click Create Account button
Expected Results	• The system does not create an account for the user • The system displays error message • Stating “This email is already associated with an account.” • The user remains on the registration popup box form.
Priority	High
	Pass: The system prevents account creation, shows the “This email is already associated with an account.” message, and keeps the user on the registration popup box. Fail: The system creates the account or does not display the “This email is already associated with an account.” error message.

ID	TC-08

Item to Test & Description	Account Login – Test the sign in process to ensure that a user can log into their account successfully
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the sign in button on the navbar. • The sign in popup box is opened.
Test Steps	1. User enters their credentials into the sign in form text fields as follows: • Email Address: [redacted] • Password: [redacted] 2. User click login button
Expected Results	• The system logs the user into their account • The system redirects the user to the dashboard page
Priority	High
Pass/Fail Criteria	Pass: The user is logged in successfully, and the system redirects them to the dashboard page. Fail: The system does not log the user in, or the user is not redirected to the dashboard after logging in.

ID	TC-09
Item to Test & Description	Account Login – Test the sign-in process to ensure that a user cannot log into their account when the email address field is left empty.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the Sign In button on the navbar.

	• The sign in popup box is opened.
Test Steps	1. User enters in their credentials: • Email Address: (Leave this field blank) • Password: [redacted] 2. User click login button
Expected Results	• The system does not log in the user • The system displays error message under the email address text field stating, “Please fill out this field.” • The user remains on the sign in popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not log in the user, displays the error message, and keeps the user on the sign-in popup box. Fail: The system does not display the error message, logs in the user, and redirects them to the dashboard page.

ID	TC-10
Item to Test & Description	Account Login – Test the sign-in process to ensure that a user cannot log into their account when the password field is left empty.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the Sign In button on the navbar. • The sign in popup box is opened.
Test Steps	1. User enters in their credentials: • Email Address: [redacted] • Password: (Leave this field blank) 2. User click login button

Expected Results	• The system does not log in the user • The system displays error message under the password text field stating, “Please fill out this field.” • The user remains on the sign in popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not log in the user, displays the error message, and keeps the user on the sign-in popup box. Fail: The system does not display the error message, logs in the user, and redirects them to the dashboard page.

ID	TC-11
Item to Test & Description	Account Login – Test the sign-in process to ensure that a user cannot log into their account when the email address and password text fields are left empty.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the Sign In button on the navbar. • The sign in popup box is opened.
Test Steps	1. User enters in their credentials: • Email Address: (Leave this field blank) • Password: (Leave this field blank) 2. User click login button
Expected Results	• The system does not log in the user • The system displays an error message under the email address field stating, “Please fill out this field,” and displays

	the same message under the password field. • The user remains on the sign in popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not log in the user, displays the error messages under both fields, and keeps the user on the sign-in popup box Fail: The system does not display the error messages, logs in the user, or redirects them to the dashboard page.

ID	TC-12
Item to Test & Description	Account Login – Test the sign-in process to ensure that a user cannot log into their account when an incorrect password is used with a correct email address.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the Sign In button on the navbar. • The sign in popup box is opened.
Test Steps	1. User enters in their credentials: • Email Address: [redacted] • Password: [redacted] 2. User click login button
Expected Results	• The system does not log in the user • The system displays an error message stating, “The password is incorrect. Please try again”. • The user remains on the sign in popup box form.

Priority	High
Pass/Fail Criteria	Pass: The system does not log in the user, displays the error message, and keeps the user on the sign-in popup box Fail: The system does not display the error message, logs in the user, or redirects them to the dashboard page.

ID	TC-13
Item to Test & Description	Account Login – Test the sign-in process to ensure that a user cannot log into their account with an unregistered email address.
Pre-Conditions	• The website and all required services are operational and running normally. • The user clicks on the Sign In button on the navbar. • The sign in popup box is opened.
Test Steps	1. User enters in their credentials: • Email Address: [redacted] • Password: [redacted] 2. User click login button
Expected Results	• The system does not log in the user • The system displays an error message stating, "User does not exist. Please try again". • The user remains on the sign in popup box form.
Priority	High
Pass/Fail Criteria	Pass: The system does not log in the user, displays the error message, and keeps the user on the sign-in popup box

	Fail: The system does not display the error message, logs in the user, or redirects them to the dashboard page.

ID	TC-14
Item to Test & Description	User Profile – Test the update email process to ensure that a user can successfully update their account email address
Pre-Conditions	• The website and all required services are operational and running normally. • The user is logged in. • The user is on the Dashboard page
Test Steps	1. The user clicks the User Profile button 2. The user locates the Update Email section and enters their new email address: • Email: [redacted] 3. User click the save changes button 4. The user enters their password in the Confirm Password popup: • Password: [redacted] 5. User click the confirm button 6. The user checks their email inbox for the verification link to verify the new email. 7. The user clicks the verification link, which opens the home page in a new browser tab and verifies the email change. 8. The user logs in with their new email address
Expected Results	• The system updates the user's current email address to their new email address.

	• Verification email link is sent to user's new email • The new email address is displayed in the current email text field on user profile popup after updating • User can login with their new email address
Priority	High
Pass/Fail Criteria	Pass: The current email address is successfully updated to the new email address. A verification email is sent, the verification link verifies the change and opens the home page in a new tab, and the user is able to log in using the new email address. Fail: The current email address is not updated to the new email address, the verification email is not sent, the verification link fails to verify the change or does not open the home page, and the user is unable to log in using the new email address.

ID	TC-15
Item to Test & Description	Manual Report – Test the manual report to ensure it generates a repair report based on the user's vehicle profile and repair item selections.
Pre-Conditions	• The website and all required services are operational and running normally. • User has a valid, verified account. • User has a vehicle profile • User is on the Dashboard page
Test Steps	1. User clicks the Manual Report button 2. User selects the saved vehicle profile from the select vehicle drop down list. 3. The user selects the area from the Area of Car drop downlist.

	4. The user selects the part from the Part drop down list. 5. The user clicks the add button to add the part to the report 6. The user clicks the Generate Report button 7. The system generates the repair report.
Expected Results	The system successfully generates a repair report based on the user's vehicle profile and repair item selections.
Priority	High
Pass/Fail Criteria	Pass: The system successfully generates a repair report based on the user's vehicle profile and repair item selections. Fail: • The system fails to generate a repair report based on the user's vehicle profile and repair item selections. • User can't select their vehicle profile • User can't select repair items for selections

ID	TC-16
Item to Test & Description	Manual Report – Test that a user can generate a repair report and download it as a PDF.
Pre-Conditions	• The website and all required services are operational and running normally. • User has a valid, verified account. • User has a vehicle profile • User is on the Dashboard page
Test Steps	1. User clicks the Manual Report button

	2. User selects the saved vehicle profile from the select vehicle drop down list. 3. The user selects the area from the Area of Car drop downlist. 4. The user selects the part from the Part drop down list. 5. The user clicks the add button to add the part to the report 6. The user clicks the Generate Report button 7. The system generates the repair report. 8. User clicks the Download button to download the PDF of the repair report.
Expected Results	• The system generates the repair report correctly with the selected vehicle and parts. • The PDF of the repair report was downloaded successfully.
Priority	High
Pass/Fail Criteria	Pass: The report is generated with correct data and the PDF downloads successfully. Fail: The report fails to generate; the PDF is missing or incomplete, or the download fails.

ID	TC-17
Item to Test & Description	Manual Report – Test that a user can delete a repair report
Pre-Conditions	• The website and all required services are operational and running normally. • User has a valid, verified account. • User has a vehicle profile • User is on the Dashboard page

Test Steps	1. User clicks the Manual Report button 2. User locates the list of previously generated repair reports. 3. User clicks the Delete button associated with the selected report. 4. The system deletes the repair report. 5. Observe that the repair report is deleted from the history section for repair reports.
Expected Results	The system deletes the selected repair report from the database. And the report is no longer displayed in the report's history section.
Priority	High
Pass/Fail Criteria	Pass: The system successfully deletes the selected report from the database, and it no longer appears in the report's history section. Fail: the system does not delete the selected report, and it remains in the report's history section.

ID	TC-18
Item to Test & Description	Smart Report – Test that the system generates a repair report using AI detection based on an uploaded vehicle image.
Pre-Conditions	• The website and all required services are operational and running normally. • User has a valid, verified account. • User has a vehicle profile • User is on the Dashboard page
Test Steps	1. User clicks the Smart Report button 2. User enters in labor rate • Labor rate: 110

	3. User clicks the Choose File button 4. User clicks the Upload & Estimate button 5. The system analyzes the image and generates a repair report using AI detection. 6. Observe that the repair report is generated in the history section.
Expected Results	The system generates a repair report based on the uploaded image and displays it in the report history section.
Priority	High
Pass/Fail Criteria	Pass: The system successfully generates a repair report based on the uploaded image, and the report appears in the history section. Fail: The system fails to generate the repair report, and the report does not get displayed in the history list.

ID	TC-19
Item to Test & Description	Smart Report – Test that the system does not generate a repair report using AI detection when no image is selected
Pre-Conditions	• The website and all required services are operational and running normally. • User has a valid, verified account. • User has a vehicle profile • User is on the Dashboard page
Test Steps	1. User clicks the Smart Report button 2. User enters in labor rate • Labor rate: “110” 3. User clicks the Upload & Estimate button

	4. Observe the generated error message that is displayed.
Expected Results	The system will display an error massage stating “At least one image is required”
Priority	High
Pass/Fail Criteria	Pass: The system displays an error message and does not generate a repair report. Fail: The system generates the repair report despite the missing labor rate, or no error message appears

ID	TC-20
Item to Test & Description	Smart Report – Test that a user can download a generated repair report as a PDF.
Pre-Conditions	• The website and all required services are operational and running normally. • User has a valid, verified account. • User has a vehicle profile • User is on the Dashboard page
Test Steps	1. User clicks the Smart Report button 2. User locates the list of previously generated repair reports. 3. User clicks the Download button associated with the selected report. 4. The system begins downloading the report as a PDF to the user’s device.
Expected Results	The selected repair report is successfully downloaded as a PDF.
Priority	High
Pass/Fail Criteria	Pass: The PDF file downloads successfully when the user clicks the Download button.

	Fail: The system does not download the PDF when the user clicks the Download button.

ID	TC-21
Item to Test & Description	Account Registration – Test the /register backend endpoint to ensure a new user can successfully create an account
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/register
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"email": [redacted]
	"password": [redacted]
	"passwordConfirm": [redacted]
	}
	8. Click send to get results

Expected Results	{
	"success": true,
	"message": "Verification email sent! Please check your inbox."
	}

Priority	High
	Pass: The test passes if Postman returns a 200 OK status code and the response contains “success”: true along with the message “Verification email sent! Please check your inbox.”

	Fail: The test fails if the system returns a 400 Bad Request status code, and the response contains “success”: false with an appropriate error message.

ID	TC-22
Item to Test & Description	Account Registration – Test the /register backend endpoint to ensure a user cannot create an account when the email address field is left empty.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/register
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"email": "",
	"password": [redacted]
	"passwordConfirm": [redacted]
	}
	8. Click send to get results

Expected Results	{
	"success": false,
	"message": "Email is required"
	}

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains

	“success”: false along with the message “Email is required” Fail: Postman returns a 200 OK status code, and the response contains “success”: true along with the message “Verification email sent! Please check your inbox.”

ID	TC-23
Item to Test & Description	Account Login – Test the /login backend endpoint to ensure a user can successfully log into their account.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/login
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	
	{
	"email": [redacted]
	"password": [redacted]
	}
	
	8. Click send to get results

Expected Results	{
	"success": true,
	"user": {
	"id": "68d6118de947a46adf018711",
	"email": [redacted]

	} }
Priority	High
Pass/Fail Criteria	Pass: Postman returns a 200 OK status code, and the response contains “success”: true along with the returned user data. Fail: The test fails if the system returns a 400 Bad Request status code, and the response contains “success”: false with an appropriate error message.

ID	TC-24
Item to Test & Description	Account Login – Test the /login backend endpoint to ensure a user cannot log into an account when the email address field is left empty.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/login
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	
	{
	"email": "",
	"password": [redacted]
	}
	
	8. Click send to get results

Expected Results	{
	"success": false,
	"message": "Email is required"
	}

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains "success": false along with the message "Email is required" Fail: Postman returns a 200 OK status code, and the response contains "success": true along with returned user data.

ID	TC-25
Item to Test & Description	Account Deactivation – Test the /deactivate-account backend endpoint to ensure the system correctly prevents account deactivation when no email address provided.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/deactivate-account
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	
	{
	"email": "",
	"password": [redacted]
	}

	8. Click send to get results

Expected Results	{
	"success": false,
	"message": "User not found"
	}

Priority	Medium
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains “success”: false along with the message “User not found” Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the message “Account deactivated successfully.”

ID	TC-26
Item to Test & Description	Account Deactivation – Test the /deactivate-account backend endpoint to ensure the system correctly prevents account deactivation when the provided email address is not registered.
Pre-Conditions	• Postman is open. • The backend server is running
Test Steps	1. Double click the Postman icon to open the program. 2. Click the new button on the ribbon and select HTTP 3. Select POST request. 4. Enter in endpoint URL: • http://localhost:5000/deactivate-account 5. Select the body tab 6. Select raw as the input and JSON as the format.

	7. Enter the JSON request test data:
	
	{
	"email": [redacted]
	"password": [redacted]
	}
	
	8. Click send to get results

Expected Results	{
	"success": false,
	"message": "User not found"
	}

Priority	Medium
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains "success": false along with the message "User not found" Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the message "Account deactivated successfully."

ID	TC-27
Item to Test & Description	Account Deactivation – Test the /deactivate-account backend endpoint to ensure the system correctly prevents account deactivation when the provided password is incorrect
Pre-Conditions	• Postman is open. • The backend server is running
Test Steps	1. Double click the Postman icon to open the program. 2. Click the new button on the ribbon and select HTTP 3. Select POST request. 4. Enter in endpoint URL:

	• http://localhost:5000/deactivate-account
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"email": [redacted]
	"password": "Hello23$a"
	}
	8. Click send to get results

Expected Results	{
	"success": false,
	"message": "Invalid password"
	}

Priority	Medium
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains "success": false along with the message "Invalid password" Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the message "Account deactivated successfully."

ID	TC-28
Item to Test & Description	Account Deactivation – Test the /deactivate-account backend endpoint to ensure the system correctly deactivates account when valid data is provided.
Pre-Conditions	• Postman is open. • The backend server is running
Test Steps	1. Double click the Postman icon to open the program.

	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/deactivate-account
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"email": [redacted]
	"password": [redacted]
	}
	8. Click send to get results

Expected Results	{
	"success": true,
	"message": "Account deactivated successfully"
	}

Priority	Medium
Pass/Fail Criteria	Pass: Postman returns a 200 OK status code, and the response contains "success": true along with the message “Account deactivated successfully.” Fail: The system returns a 400 Bad Request status code, and the response contains “success”: false with an appropriate error message.

ID	TC-29
Item to Test & Description	Account Deletion – Test the /delete-account backend endpoint to ensure the system correctly prevents account deletion when the provided email address is not registered.
Pre-Conditions	• Postman is open.

	• The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/delete-account
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"email": [redacted]
	"password": [redacted]
	}
	8. Click send to get results

Expected Results	{
	"success": false,
	"message": "User not found"
	}

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains "success": false along with the message "User not found" Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the message "Account deleted successfully."

ID	TC-30
Item to Test & Description	Account Deletion – Test the /delete-account backend endpoint to ensure the system

	correctly prevents account deletion when the provided password is incorrect
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/delete-account
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"email": [redacted]
	"password": "helsoee12$s"
	}
	8. Click send to get results

Expected Results	{
	"success": false,
	"message": "Invalid password"
	}

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains “success”: false along with the message “Invalid password” Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the message “Account deleted successfully.”

ID	TC-31

Item to Test & Description	Account Deletion – Test the /delete-account backend endpoint to ensure the system correctly deletes the account when valid data is provided.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/delete-account
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"email": [redacted]
	"password": [redacted]
	}
	8. Click send to get results

Expected Results	{
	"success": true,
	"message": "Account deleted successfully."
	}

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 200 OK status code, and the response contains "success": true along with the message “Account deleted successfully.” Fail: The system returns a 400 Bad Request status code, and the response contains “success”: false with an appropriate error message.

ID	TC-32
Item to Test & Description	Vehicle Profile – Test the /api/vehicleinfo backend endpoint to ensure the system correctly creates a vehicle profile.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/api/vehicleinfo
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	{
	"vin": "JTENU5JR7D1234567",
	"email": [redacted]
	}
	8. Click send to get results

Expected Results	{ "message": "2013 Toyota 4runner added successfully!", "source": "NinjaAPI", "vehicle": { "email": [redacted] "vin": "JTENU5JR7D1234567", "nickname": "2013 Toyota 4runner", "year": "2013", "make": "Toyota", "model": "4runner", "trim": "", "engine": "4L 6-Cyl", "fuel": "Gasoline", "drivetrain": "RWD", "transmission": "a", "mpg_city": "17", "mpg_highway": "22", "class": "Unknown", "data": { "vin": "JTENU5JR7D1234567", "country": "Japan", "manufacturer": "Toyota", "model": "4Runner", "class": "", "region": "Asia", "wmi": "JTE", "vds": "NU5JR7", "vis": "D1234567", "year": "2013", "engine": "4L 6-Cyl", "fuel_type": "gas", "drive": "rwd", "transmission": "a", "city_mpg": 17, "highway_mpg": 22 }, "_id": "691a044df2f84cde14f38359", "createdAt": "2025-11-16T17:05:17.173Z", "updatedAt": "2025-11-16T17:05:17.173Z", "__v": 0 } }

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 200 OK status code, and the response contains "success": true along with the returned vehicle information. Fail: The system returns a 400 Bad Request status code, and the response contains "success": false with an appropriate error message.

ID	TC-33
Item to Test & Description	Vehicle Profile – Test the /api/vehicleinfo backend endpoint to ensure the system correctly prevents creating a vehicle profile when no VIN is provided.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/api/vehicleinfo
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	
	{
	"vin": "",
	"email": [redacted]
	}
	8. Click send to get results

Expected Results	{ "message": "VIN and email required"

	}
Priority	High
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains “success”: false along with the message “VIN and email required” Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the returned vehicle information.

ID	TC-34
Item to Test & Description	Vehicle Profile – Test the /api/vehicleinfo backend endpoint to ensure the system correctly prevents creating a vehicle profile when no email address is provided.
Pre-Conditions	• Postman is open. • The backend server is running

Test Steps	1. Double click the Postman icon to open the program.
	2. Click the new button on the ribbon and select HTTP
	3. Select POST request.
	4. Enter in endpoint URL:
	• http://localhost:5000/api/vehicleinfo
	5. Select the body tab
	6. Select raw as the input and JSON as the format.
	7. Enter the JSON request test data:
	
	{
	"vin": " JTENU5JR7D1234567",
	"email": ""
	}
	8. Click send to get results
	9. Observe the results in the response section

Expected Results	{
	"message": "VIN and email required"
	}

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 400 bad Request status code, and the response contains “success”: false along with the message “VIN and email required” Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the returned vehicle information.

ID	TC-35
Item to Test & Description	Vehicle Profile – Test the /api/history backend endpoint to ensure the system correctly returns the vehicle profile.
Pre-Conditions	• Postman is open. • The backend server is running
Test Steps	1. Double click the Postman icon to open the program. 2. Click the new button on the ribbon and select HTTP 3. Select GET request. 4. Enter in endpoint URL: • http://localhost:5000/api/history 5. Select the params tab 6. In the first row of the key column, enter: email 7. Select value column, first row and enter in email address: • [redacted] 9. Click send to get results

Expected Results	JSON array returns vehicle records with metadata, VIN details, manufacturer, year, and specs.
Priority	High
Pass/Fail Criteria	Pass: Postman returns a 200 OK status code, and the response contains "success": true along with the returned vehicle information. Fail: The system returns a 400 Bad Request status code, and the response contains "success": false with an appropriate error message.

ID	TC-36
Item to Test & Description	Vehicle Profile – Test the /api/history backend endpoint to ensure the system correctly prevents the return of the vehicle profile when no email address is provided.
Pre-Conditions	• Postman is open. • The backend server is running
Test Steps	1. Double click the Postman icon to open the program. 2. Click the new button on the ribbon and select HTTP 3. Select GET request. 4. Enter in endpoint URL: • http://localhost:5000/api/history 5. Select the params tab 6. In the first row of the key column, enter: email. 7. In the first row of the value column, leave the field blank 8. Click send to get results

Expected Results	{
	"message": "email is required"
	}

Priority	High
Pass/Fail Criteria	Pass: Pass: Postman returns a 400 bad Request status code, and the response contains “success”: false along with the message “email is required” Fail: Postman returns a 200 OK status code, and the response contains "success": true along with the returned vehircle information.

ID	TC-37
Item to Test & Description	Vehicle Profile – Test the /api/history/id backend endpoint to ensure the system correctly prevents deleting the vehicle profile when id is incorrect.
Pre-Conditions	• Postman is open. • The backend server is running
Test Steps	1. Double click the Postman icon to open the program. 2. Click the new button on the ribbon and select HTTP 3. Select DELETE request. 4. Enter in endpoint URL: • http://localhost:5000/api/history/id 5. Click send to get results

Expected Results	{
	"ok": **false**
	}

Priority	High

Pass/Fail Criteria	Pass: The system returns a 400 Bad Request status code, and the response contains “ok”: false indicating deletion failed Fail: Postman returns a 200 OK status code, and the response contains "ok": true, indicating successful deletion

ID	TC-38
Item to Test & Description	Vehicle Profile – Test the /api/history/id backend endpoint to ensure the system correctly deletes the vehicle profile.
Pre-Conditions	• Postman is open. • The backend server is running
Test Steps	1. Double click the Postman icon to open the program. 2. Click the new button on the ribbon and select HTTP 3. Select DELETE request. 4. Enter in endpoint URL: • http://localhost:5000/api/history/id 5. Click send to get results

Expected Results	{
	"ok": **true**
	}

Priority	High
Pass/Fail Criteria	Pass: Postman returns a 200 OK status code, and the response contains "ok": true, indicating successful deletion

	Fail: The system returns a 400 Bad Request status code, and the response contains “ok”: false indicating deletion failed

Unit Test Cases List:

ID	TC-39
Item to Test & Description	Forgot Password – Test the handlePassReset function to ensure it alerts the user when no email is entered.
Pre-Conditions	• The handlePassReset function is imported into a separate file for testing. • Axios.post request is mocked but is not called • handlePassReset required callback parameters are mocked
Test Steps	1. Leave the email argument empty 2. Call the handlePassReset with no email and mocked data. 3. Verify axios.post was not triggered 4. Verify setMessage was triggered for a failure message.
Expected Results	The function displays an alert with the message” Please enter your email first!”
Priority	low
Pass/Fail Criteria	Pass: The function displays an alert with the message” Please enter your email first!” and axios.post was never called. Fail: The function does not display an alert with the message” Please enter your email first!” and axios.post was called.

ID	TC-40
Item to Test & Description	Forgot Password – Test the handlePassReset function to ensure it sends the user a reset password link.
Pre-Conditions	• The handlePassReset function is imported into a separate file for testing. • Axios post request is mocked for a successful response • handlePassReset parameters are mocked
Test Steps	1. Mock axios.post to succeed 2. Call the handlePassReset with valid email and mocked data. 3. Verify axios.post was triggered 4. Verify setMessage was triggered for a success message.
Expected Results	the function sets a success message such as “Reset link sent!” and performs the expected success actions
Priority	low
Pass/Fail Criteria	Pass: The function sends a rest password link to the user email. Fail: The function does not send a rest password link to the user email.

ID	TC-41
Item to Test & Description	Forgot Password – Test the handlePassReset function to ensure it does not send the user a reset password link because of failure with the API.

Pre-Conditions	• The handlePassReset function is imported into a separate file for testing. • Axios.post request is mocked for failure • handlePassReset parameters are mocked
Test Steps	1. Mock axios.post to reject with an error 2. Call the handlePassReset 3. Verify that setMessage was triggered with the expected error message.
Expected Results	The function displays an error message stating “Error sending reset request”.
Priority	low
Pass/Fail Criteria	Pass: The function sets the error message to “Error sending reset request”. And does not send the post request. Fail: The function does not set the error message to “Error sending reset request”. And does send the post request.

ID	TC-42
Item to Test & Description	Account Deletion – Test the handleDeleteAccount function to ensure it successfully deletes a user account.
Pre-Conditions	• The handleDeleteAccount function is imported into a separate file for testing. • Axios.post request is mocked. • handleDeleteAccount callback parameters are mocked

Test Steps	1. Mock axios.post as success with the value: true. 2. Mock the argument of email and password. • Email: [redacted] • Password: [redacted] 3. Trigger the handleDeleteAccount with mocked data. 4. Verify that • the message of “account deleted successfully” is called • User data is deleted parameter is called • Redirect user parameter is called
Expected Results	The function successfully deletes the user account, clears stored user data, displays the message: “Account deleted successfully.”, and redirects the user to the home page.
Priority	low
Pass/Fail Criteria	Pass: The function successfully deletes the user account, clears stored user data, displays the message: “Account deleted successfully.”, and redirects the user to the home page. Fail: The function does not display a success message stating: “Account deleted successfully.” And the user data does not get deleted, and the user stays on the delete account popup box.

ID	TC-43
Item to Test & Description	Account Deletion – Test the handleDeleteAccount function to ensure it

	does not delete a user account if the backend rejects the request (e.g., password is empty).
Pre-Conditions	• The handleDeleteAccount function is imported into a separate file for testing. • Axios.post request is mocked • handleDeleteAccount callback parameters are mocked
Test Steps	1. Mock axios.post to reject with an error 2. Mock axios.post as success with the value: true. 3. Mock the argument of email and password. • Email: [redacted] • Password: “” 4. Trigger the handleDeleteAccount with mocked data. 5. Verify that • the message of “enter password” is called. • user data deletion callback is not called. • The redirect callback is not called.
Expected Results	The function does not delete the user account, does not clear stored data, displays an error message, and the user remains on the delete account popup.
Priority	low
Pass/Fail Criteria	Pass: The test passes if the backend rejects the request, the error message is displayed, the user data is not removed, and the user is not redirected. Fail: The test fails if the backend accepts the request, the success message is displayed, the user data is removed, or the user is redirected.

ID	TC-44
Item to Test & Description	Account Deletion – Test the handleDeleteAccount function to ensure it does not delete a user account because of API failure.
Pre-Conditions	• The handleDeleteAccount function is imported into a separate file for testing. • Axios.post request is mocked • handleDeleteAccount callback parameters are mocked
Test Steps	1. Mock axios.post to reject with an error. 2. Mock the argument of email and password. • Email: [redacted] • Password: [redacted] 3. Trigger the handleDeleteAccount with mocked data. 4. Verify that 5. The error message of “enter password” is called. 6. user data deletion callback is not called. 7. The redirect callback is not called.
Expected Results	The function displays an error message stating “Error deleting account”.
Priority	low
Pass/Fail Criteria	Pass: The function sets the error message to “Error deleting account”. And the user data does not get deleted, and the user stays on the delete account popup box

	Fail: The function does not set the error message to “Error deleting account”. And the user data does get deleted, and the user gets redirected to the home page.

System Test Cases List

ID	TC-45
Item to Test & Description	End-to-end system flow — Verify that a user can register, verify their email, log in, create a vehicle profile, generate a manual repair report, and download it as a PDF.
Pre-Conditions	• The backend server is running • The frontend is running • User have a valid email address • TC-1 through TC-31 have passed.
Test Steps	1. User clicks the register button in the navbar 2. enter user credentials: • Email • Password • Confirm password 3. Open user email inbox 4. User clicks the verification link sent by TrueRepair 5. User is redirected to the home page with a notification stating “Account verified.” 6. User clicks the login button. 7. User enters credentials: • Email • password 8. User clicks the login button 9. User clicks the My Vehicle button on the dashboard page 10. User enter in their vehicle VIN #

	11. User clicks the Search & Save button to decode the vin and the vehicle profile 12. User clicks the close button to close the My Vehicle popup. 13. User clicks the Manual Report button on the dashboard. 14. User selects the saved vehicle profile from the select vehicle drop down list. 15. The user selects the area from the Area of Car drop downlist. 16. The user selects the part from the Part drop down list. 17. The user clicks the add button to add the part to the report 18. The user clicks the Generate Report button 19. The system generates the repair report. 20. The user clicks the Download button to download the generated PDF repair report.
Expected Results	• The account is successfully created • Email verification link redirects the user and displays "Account verified." • The login is successful • The user gets directed to the dashboard • The system decodes the VIN and successfully saves the vehicle profile. • The manual repair report is generated based on the selected vehicle profile and repair items. • The report downloads successfully as a PDF file.
Priority	High
Pass/Fail Criteria	Pass: The user can successfully register, verify their email, log in, create a vehicle profile, generate a repair report, and download the PDF without any errors.

	Fail: If any of the following occur: • Account cannot be created or verified. • Login fails with valid credentials. • Vehicle profile cannot be saved. • The repair report cannot be generated. • The PDF report fails to download.

ID	TC-46
Item to Test & Description	End-to-end system flow — Verify that a user can log in, create a vehicle profile, generate AI repair report, and download it as a PDF using Smart Report.
Pre-Conditions	• The backend server is running • The frontend is running • User has a valid, verified account. • TC-1 through TC-31 has passed.
Test Steps	1. User clicks the login button. 2. User enters credentials: • Email: [redacted] • Password: [redacted] 3. User clicks the login button 4. User clicks the My Vehicle button on the dashboard page 5. User enter in their vehicle VIN # 6. User clicks the Search & Save button to decode the vin and the vehicle profile 7. User clicks the close button to close the My Vehicle popup. 8. User clicks the Smart Report button on the dashboard. 9. User enters in labor rate • Labor rate: 110 10. User clicks the Choose File button

	11. User clicks the Upload & Estimate button 12. The system generates the repair report using AI to detect parts. 13. The user clicks the Download button to download the generated PDF repair report
Expected Results	• The login is successful • The user gets directed to the dashboard • The system decodes the VIN and successfully saves the vehicle profile. • The repair report is generated based on the image the user uploaded. • The report downloads successfully as a PDF file.
Priority	High
Pass/Fail Criteria	Pass: The user can successfully log in, create a vehicle profile, generate a repair report using AI, and download the PDF without any errors. Fail: If any of the following occur: • Login fails with valid credentials. • Vehicle profiles cannot be saved. • The repair report cannot be generated. The PDF report fails to download.

ID	TC-47
Item to Test & Description	End-to-end system flow — Verify that a user can log in, create a vehicle profile, deactivate their account, reactivate it, and permanently delete their account.
Pre-Conditions	• The backend server is running • The frontend is running

	• User has a valid, verified account. • TC-1 through TC-31 has passed.
Test Steps	1. User clicks the Login button on the navbar. 2. User enters valid credentials and logs in (email, password). 3. User clicks My Vehicle on the dashboard. 4. User enters a valid VIN and clicks Search & Save to create a vehicle profile. 5. User closes the My Vehicle popup. 6. User opens the Settings page. 7. User selects Deactivate Account. 8. System displays a password confirmation popup. 9. User enters their password and confirms deactivation. 10. System deactivates the user account and redirects them to the homepage. 11. User logs in again with email and password to reactivate their account. 12. User clicks the My Vehicle popup on the Dashboard to verify their saved vehicle profile still exists. 13. User clicks the close button to close the vehicle profile popup. 14. User selects Delete Account from the Settings page. 15. System displays a password confirmation popup. 16. User enters their password and confirms deletion by clicking the delete account button. 17. System deletes the user account. 18. System redirects the user to the Homepage
Expected Results	• User successfully logs in • The user gets directed to the dashboard

	• Vehicle profile is successfully created and saved. • Account is successfully deactivated. • Account is automatically reactivated upon login. • User data remains intact after reactivation (vehicle profile still exists). • Account is permanently deleted when confirmed.
Priority	High
Pass/Fail Criteria	Pass: The user is able to log in, create a vehicle profile, deactivate and reactivate their account, and successfully delete their account. All expected system behaviors occur exactly as described. Fail: If any of the following occur: • Login fails with valid credentials. • Vehicle profiles cannot be saved. • Account cannot be deactivated or reactivated. • Vehicle profile is lost after reactivation. • User cannot delete their account

ID	TC-48
Item to Test & Description	Homepage Load Time – Test that the homepage displays within an acceptable time limit (under 5 seconds).
Pre-Conditions	The homepage is accessible and opened in the browser.
Test Steps	1. Right-click anywhere on the page 2. Select DevTools (Inspect)

	3. Locate the network tab 4. Click network tab 5. Reload the page. 6. Record the Load time from the bottom of the Network panel.
Expected Results	The page fully loads under 5 seconds
Priority	medium
Pass/Fail	Pass: The homepage loads within the 5-second time limit. Fail: The homepage takes longer than 5 seconds to fully load.

Acceptance Test cases:

ID	TC-49
Item to Test & Description	Complete user workflow — ensure a user can create a verified account, log in, create a vehicle profile, generate a manual repair report, and download it as a PDF.
Pre-Conditions	• Frontend is running • Backend server is running • User has an access their email inbox
Test Steps	1. User register a new account 2. Users open their email inbox and clicks the verification link. 3. User logs in their account 4. User create a vehicle profile (Manually or VIN) 5. User opens Manual Report popup 6. User selects a saved vehicle profile 7. User adds repair item options

	8. User generates the manual repair report 9. User downloads the manual repair report as a PDF
Expected Results	• The account gets created • user logs in successfully • vehicle profile is saved correctly • manually repair report generated correctly • Repair report is successfully downloaded as a PDF
Priority	High
Pass/Fail Criteria	Pass: The user successfully completes registration, login, vehicle profile creation, repair report generation, and PDF download. Fail: Any step-in registration, login, vehicle profile creation, report generation, or PDF download fails

ID	TC-50
Item to Test & Description	Complete user workflow — ensure a user can log in, create a vehicle profile, generate a Smart Report with AI, and download the report as a PDF.
Pre-Conditions	• Frontend is running • Backend server is running • User has a verified account
Test Steps	

	1. User logs in their account 2. User create a vehicle profile (Manually or VIN) 3. User opens Smart Report popup 4. User selects a saved vehicle profile 5. User provides a labor rate 6. User uploads an image 7. The system analyzes the image and generates the repair report 8. User downloads the repair report as a PDF
Expected Results	• user logs in successfully • vehicle profile is saved correctly • Repair report is generated based on AI detections • Repair report is successfully downloaded as a PDF
Priority	High
Pass/Fail Criteria	Pass: The user successfully logs in, creates a vehicle profile, generates the AI-based repair report, and downloads the report as a PDF. Fail: Any step-in login, vehicle profile creation, AI report generation, or PDF download fails.

16 Roles and Responsibilities

To successfully conduct testing for the TrueRepairs application, the development team assigns each member a specific role with clearly defined responsibilities. These roles ensure that test

creation, execution, and documentation are completed accurately and efficiently. The assigned roles include the Test Lead, Gray Box Testers, and Unit Tester.

The test lead: Hisham
The Test Lead is responsible for supervising all testing activities for the TrueRepair application. This includes ensuring that all tests are executed according to the test schedule and that results are properly documented. The Test Lead will also verify that the traceability matrix remains aligned with the SRS and SDS requirements. Additionally, the Test Lead is responsible for executing integration tests when needed.

Gray box tester: Jibran, Robert
The Gray Box Testers are responsible for creating and executing the majority of integration and system test cases. They document the results of each test and track errors if they occur. Their role ensures proper validation of the application's workflows, UI behavior, and main functionality with api integrations.

Unit tester: Ishmael
The Unit Tester focuses primarily on creating and executing unit tests based on the defined unit test cases. They are also responsible for recording all unit test results. When required, the Unit Tester may assist with executing integration tests and documenting those outcomes as well.

17 Test Report

Test Case	Test Executor	Pass or Fail	Date	Additional Notes
TC-01	Ishmael	Pass	10/10	
TC-02	Ishmael	Pass	10/10	
TC-03	Ishmael	Pass	10/10	
TC-04	Robert	Pass	10/11	
TC-05	Jibran	Pass	10/11	
TC-06	Hisham	Pass	10/12	
TC-07	Robert	Pass	10/13	
TC-08	Hisham	Pass	10/13	
TC-09	Jibran	Pass	10/14	

TC-10	Hisham	Pass	10/15	
TC-11	Ishamel	Pass	10/17	
TC-12	Robert	Pass	10/17	
TC-13	Ishmael	Pass	10/17	
TC-14	Hisham	Pass	10/18	
TC-15	Ishmael	Pass	10/18	
TC-16	Robert	Pass	10/19	
TC-17	Jibran	Pass	10/20	
TC-18	Hisham	Pass	10/21	
TC-19	Jibran	Pass	10/22	
TC-20	Hisham	Pass	10/23	
TC-21	Ishamel	Pass	10/25	
TC-22	Robert	Pass	10/25	
TC-23	Ishmael	Pass	10/26	
TC-24	Hisham	Pass	10/26	
TC-25	Ishmael	Pass	10/26	
TC-26	Robert	Pass	10/27	
TC-27	Jibran	Pass	10/28	
TC-28	Hisham	Pass	10/29	
TC-29	Jibran	Pass	10/29	
TC-30	Hisham	Pass	10/30	
TC-31	Hisham	Pass	11/01	
TC-32	Ishamel	Pass	11/02	
TC-33	Robert	Pass	11/02	
TC-34	Ishmael	Pass	10/02	

TC-35	Hisham	Pass	10/03	
TC-36	Ishamel	Pass	10/03	
TC-37	Robert	Pass	10/04	
TC-38	Ishmael	Pass	11/05	
TC-39	Hisham	Pass	11/07	
TC-40	Ishmael	Pass	11/09	
TC-41	Robert	Pass	11/11	
TC-42	Jibran	Pass	11/13	
TC-43	Hisham	Pass	11/14	
TC-44	Jibran	Pass	11/15	
TC-45	Hisham	Pass	11/16	
TC-46	Ishamel	Pass	11/18	
TC-47	Robert	Pass	11/20	